

Java

Classi astratte

Classi astratte

- L'ereditarietà porta a riflettere sul rapporto fra *progetto* e *struttura*: una classe può limitarsi a *definire solo l'interfaccia*... lasciando “in bianco” uno o più metodi... ..che verranno poi implementati dalle classi derivate
- Una classe astratta:
 - fattorizza, *dichiarandole*, operazioni comuni a tutte le sue sottoclassi... .. ma non le *definisce* (implementa)
- In effetti, non viene creata *per definire istanze* (che non saprebbero come rispondere ai metodi “lasciati in bianco”) **ma per derivarne altre classi**, che dettaglieranno i metodi qui solo dichiarati.

Definizione di Classi Astratte

- Una classe astratta è simile a una classe regolare
 - Ha una struttura simile
 - Può avere attributi
 - tipi primitivi
 - classi (static)
 - istanze di oggetti
 - Può avere metodi
- Frequentemente una classe astratta ha almeno un metodo che è dichiarato ma non implementato

Classi astratte

■ Utilità:

■ Serve come una superclasse

- oltre che le superclassi regolari una classe astratta può implementare dei metodi la cui implementazione è demandata alle sottoclassi
- è un modo per imporre l'esistenza del metodo in una classe derivata

■ Possono essere definiti riferimenti

■ Limitazioni:

■ Non può essere istanziato un oggetto di tale classe e quindi modella concetti astratti

■ Non possono essere istanziati oggetti poichè non è totalmente definito il loro comportamento

Classi astratte

- Una classe astratta può (ma non deve) contenere metodi astratti cioè non implementati.
- Una classe contenente metodi astratti deve essere astratta.
- Una classe derivata da una classe astratta che override tutti i metodi astratti può (ma non deve) essere astratta
- Una classe derivata da una classe astratta che non effettua l'override di tutti i metodi astratti deve essere astratta

Classi astratte: esempio

```
public abstract class Animale {  
    public abstract String verso();  
    public abstract String si_muove();  
    public abstract String vive();  
    ...  
}
```

- I metodi astratti *non hanno corpo*
- Se *anche solo un metodo* è abstract, la classe deve essere qualificata abstract, altrimenti si ha **ERRORE**.

Classi astratte: perchè

Esempio: gli animali

- parlare di “animali” ci è molto utile, però *non esiste “il generico animale” !!*
Nella realtà esistono solo *animali specifici* ma la categoria concettuale “animale” ci fa molto comodo per “parlare del mondo”
- Tutti sappiamo cosa sia un animale:
 - ogni animale ha *un qualche verso*
 - ogni animale si muove *in qualche modo*
 - ogni animale vive *in un qualche ambiente*
- ma *proprio per questo, non esiste “il generico animale”!*
- Ogni animale reale:
 - ha *uno specifico verso*,
 - si muove *in uno specifico modo*,
 - vive *in uno specifico ambiente*. (acqua, aria, terraferma, etc.)

Classi astratte: esempio

```
public abstract class Animale {  
    public abstract String verso();  
    public abstract String  
si_muove();  
    public abstract String vive();  
    ...  
}
```

Esprime il fatto che ogni animale ha un verso, si muove, e vive da qualche parte..

...ma in generale non si può dire come.

Classi astratte: esempio

```
public abstract class AnimaleTerrestre extends Animale
{
    public String vive() { // era abstract
        return "sulla terraferma";
    }
    ...
}
```

Una classe derivata può definire uno o più metodi che erano astratti se *anche solo un metodo* rimane astratto, la classe derivata è comunque *astratta* (e deve essere qualificata come tale)

Esempio completo

- Ipotesi:

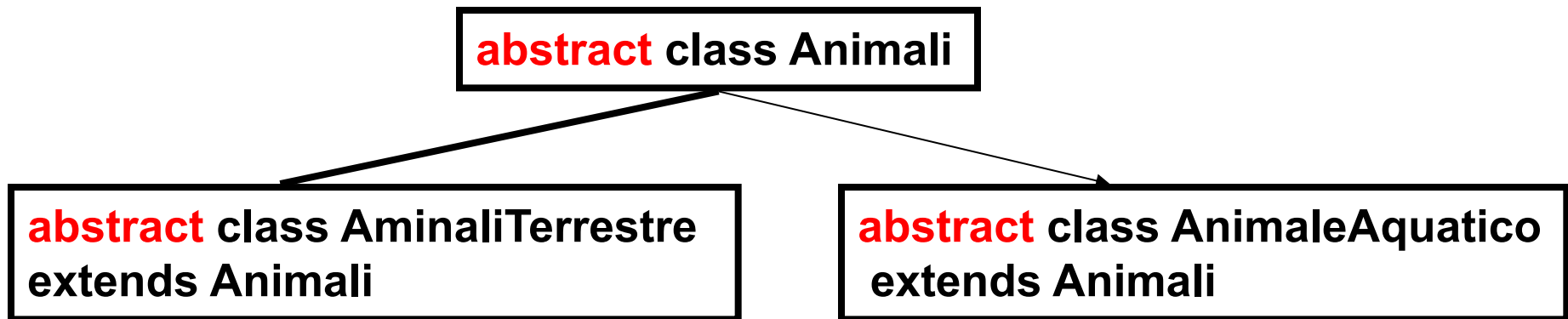
- ogni animale ha un metodo `chi_sei()` che restituisce una stringa descrittiva
- ogni animale ha un metodo `mostra()` che lo stampa a video e che è *indipendente* dallo specifico animale (si appoggia sugli altri metodi)
- tutti gli animali si rappresentano nello stesso modo, tramite il loro *nome* e il *verso* che fanno.

Esempio completo

```
public abstract class Animale {  
    private String nome;  
    protected String verso;  
    public Animale(String s) { nome=s; }  
    public abstract String si_muove();  
    public abstract String vive();  
    public abstract String chi_sei();  
    public void mostra() {  
        System.out.println(nome + ", " +  
            chi_sei() + ", " + verso +", si muove " +  
            si_muove() + " e vive " + vive() );  
    }  
}
```

Esempio completo

- Una possibile classificazione:



- Sono ancora classi astratte:
 - nulla si sa del movimento
 - quindi è impossibile definire il metodo `si_muove()`

Esempio completo

```
public abstract class AnimaleTerrestre extends Animale
{
    public AnimaleTerrestre(String s) {super(s); }
    public String vive() {
        return "sulla terraferma";
    }
    public String chi_sei() {
        return "un animale terrestre";
    }
}
```

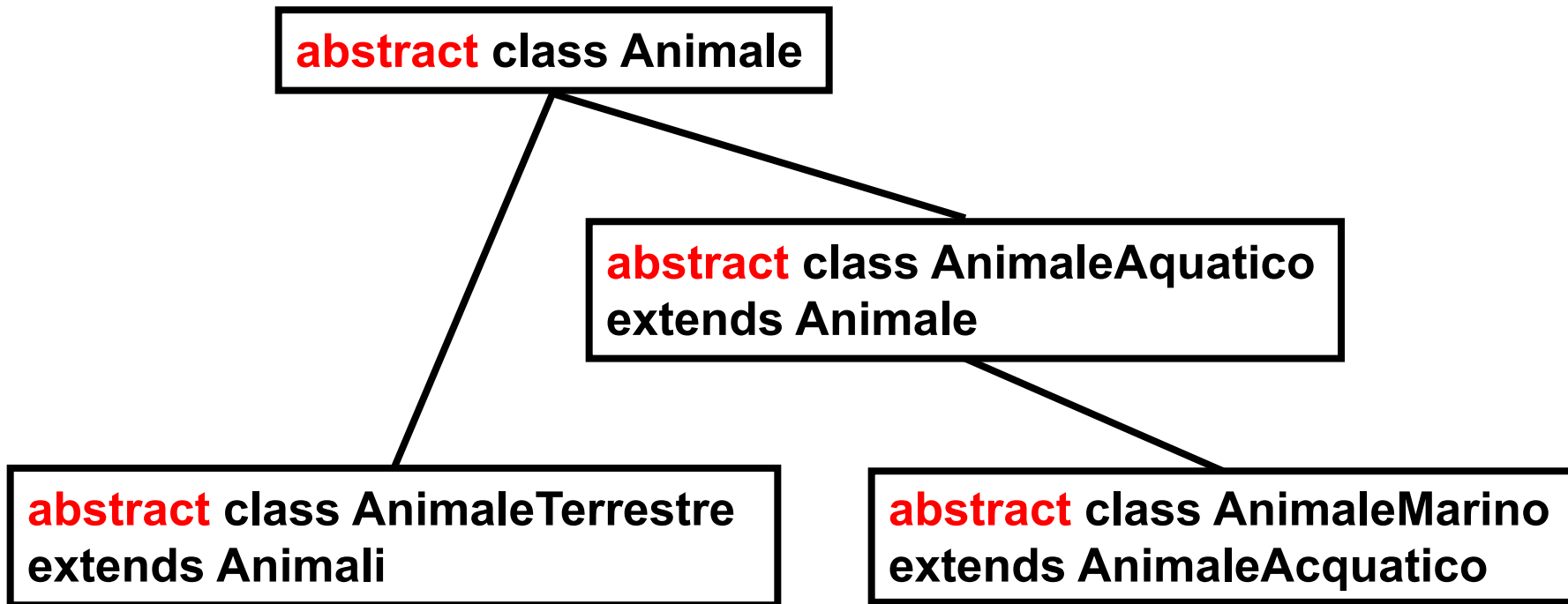
- Due metodi astratti su tre sono ridefiniti, ma ***uno è ancora astratto quindi*** la classe è ancora astratta

Esempio completo

```
public abstract class AnimaleAcquatico extends Animale
{
    public AnimaleAcquatico(String s) {super(s); }
    public String vive() {
        return "nell'acqua";
    }
    public String chi_sei() {
        return "un animale acquatico";
    }
}
```

- Due metodi astratti su tre sono ridefiniti, ma ***uno è ancora astratto*** la classe è ancora astratta

Esempio completo



■ Una possibile specializzazione:

- Perché introdurre l'animale marino?
- non è correlato a verso, movimento, ma rispecchia semplicemente una realtà.

Esempio completo

```
public abstract class AnimaleMarino extends AnimaleAcquatico {  
  
    public AnimaleMarino(String s) {super(s); }  
    public String vive() {  
        return "in mare";  
    }  
    public String chi_sei() {  
        return "un animale marino";  
    }  
}
```

- Specializza i metodi `vive()` e `chi_sei()`, ma ***non definisce l'altro e quindi*** è ancora astratta

Esempio completo

- ▣ LA TASSONOMIA COMPLETA

- ▣ ANIMALE

- ▣ ANIMALE_TERRESTRE

- ▣ QUADRUPEDE

- ▣ CAVALLO

- ▣ BIPEDE

- ▣ UOMO

- ▣ UCCELLO

- ▣ CORVO

- ▣ PINGUINO

- ▣ ANIMALE_ACQUATICO

- ▣ ANIMALE_MARINO

- ▣ PESCE

- ▣ TONNO

Le classi “concrete”

```
public class PesceDiMare extends AnimaleMarino {  
    public PesceDiMare(String s) {  
        super(s);  
        verso = "non fa versi";  
    }  
    public String chi_sei() {  
        return "un pesce (di mare)";  
    }  
    public String si_muove() {  
        return "nuotando";  
    }  
}
```

- Definisce l'ultimo metodo astratto rimasto `si_muove()`. La classe non è più astratta

Le classi “concrete”

```
public class Uccello extends AnimaleTerrestre {  
    public Uccello(String s){  
        super(s);  
        verso="cinguetta"; }  
    public String si_muove(){  
        return "volando";  
    }  
    public String chi_sei(){  
        return "un uccello";  
    }  
    public String vive(){  
        return "in un nido su un albero";  
    }  
}
```

Le classi “concrete”

```
public class Bipede extends AnimaleTerrestre {  
    public Bipede(String s){  
        super(s);  
    }  
    public String si_muove(){  
        return "avanzando su 2 zampe";  
    }  
    public String chi_sei(){  
        return "un animale con due zampe";  
    }  
}
```

Le classi “concrete”

```
public class Quadrupede extends AnimaleTerrestre {  
    public Quadrupede(String s){  
        super(s);  
    }  
    public String si_muove(){  
        return "avanzando su 4 zampe";  
    }  
    public String chi_sei(){  
        return "un animale con 4 zampe";  
    }  
}
```

Le classi “concrete”

```
public class Cavallo extends Quadrupede{  
    public Cavallo(String s){  
        super(s);  
        verso = "nitrisce";  
    }  
    public String chi_sei(){  
        return "un cavallo";  
    }  
}
```

Le classi “concrete”

```
public class Corvo extends Uccello {  
    public Corvo(String s){  
        super(s);  
        verso = "gracchia";  
    }  
    public String chi_sei(){  
        return "un corvo";  
    }  
}
```

Le classi “concrete”

```
public class Uomo extends Bipede {  
    public Uomo(String s){  
        super(s);  
        verso = "parla";  
    }  
    public String si_muove(){  
        return "camminando su 2 gambe";  
    }  
    public String chi_sei(){  
        return "un homo sapiens";  
    }  
    public String vive(){  
        return "in condominio";  
    }  
}
```

Altre classi più specifiche

```
public class Pinguino extends Uccello{  
    public Pinguino(String s){  
        super(s);  
        verso = "non fa versi";  
    }  
    public String chi_sei(){  
        return "un pinguino";  
    }  
    public String si_muove(){  
        return "ma non sa volare";  
    }  
}
```

Altre classi più specifiche

```
public class Tonno extends PesceDiMare{  
    public Tonno(String s){  
        super(s);  
    }  
    public String chi_sei(){  
        return "un tonno";  
    }  
}
```

Un main

UN MAIN... “mondo di animali”

```
public class MondoAnimale {  
    public static void main(String args[]) {  
        Cavallo c = new Cavallo("Furia");  
        Uomo h = new Uomo("John");  
        Corvo w = new Corvo("Pippo");  
        Tonno t = new Tonno("Giorgio");  
        Uccello u = new Uccello("Gabbiano");  
        Pinguino p = new Pinguino("Tweety");  
        c.mostra(); h.mostra();  
        w.mostra(); t.mostra();  
        u.mostra(); p.mostra();  
    }  
}
```